

```
import pandas as pd
data = ([1,2,3,4])
s = pd.Series(data)
print(s)
```

```
0    1
1    2
2    3
3    4
dtype: int64
```

```
import pandas as pd
data=([1,2,3,4])
s = pd.Series(data)
print(s)
a = pd.Series(data,index=["a","b","c","d"])
print(a)
s["b"] = 90
print(s)
```

```
0    1
1    2
2    3
3    4
dtype: int64
a    1
b    2
c    3
d    4
dtype: int64
0    1
1    2
2    3
3    4
b    90
dtype: int64
```

```
#updating the series
d = [1,2,3,4,5]
s=pd.Series(d)
print(s)
s[2]=90
print(s)
#inserting new value with index and value
s[6] = 60
print(s)
#delete
s=s.drop(1)
print(s)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
0    1
1    2
2    90
3    4
4    5
dtype: int64
0    1
1    2
2    90
3    4
4    5
6    60
dtype: int64
0    1
2    90
3    4
4    5
6    60
dtype: int64
```

```
#inserting range of data
data = [11,12,13,14,15]
s = pd.Series(data,index=[0,1,2,3,4])
new_data = pd.Series([100,200,300], index=[5,6,7])
s=pd.concat([s,new_data])
print(s)
```

```

0    11
1    12
2    13
3    14
4    15
5   100
6   200
7   300
dtype: int64

```

```

data = [11,12,13,14,15]
s = pd.Series(data,index=[0,1,2,3,4])
new_data = pd.Series([100,200,300], index=[5,6,7])
s=pd.concat([s,new_data])
print(s)
#delete range of series
s=s.drop(s.index[0:3])
print(s)
#entire series to remove
del s

```

```

0    11
1    12
2    13
3    14
4    15
5   100
6   200
7   300
dtype: int64
3    14
4    15
5   100
6   200
7   300
dtype: int64

```

```

#student marks
#student_id = [101,102,103,104,105]
#pandas series using student id as the index
marks = [78,85,92,67,88]
s= pd.Series(marks,index=[101,102,103,104,105])
print(s)
#disp marks of stude 103
print(s[103])
#change the marks of stdu 104 from 67 to 75
s[104] = 75
print(s)
#delete stude 102 from series
s=s.drop(102)
print(s)
#add new stude 106 with marks = 90
new = pd.Series([90],index=[106])
s=pd.concat([s,new])
print(s)

```

```

101    78
102    85
103    92
104    67
105    88
dtype: int64
92
101    78
102    85
103    92
104    75
105    88
dtype: int64
101    78
103    92
104    75
105    88
dtype: int64
101    78
103    92
104    75
105    88

```

```
106    90
dtype: int64
```

```
import pandas as pd
data = {"a":1001,"b":1002,"c":1003,"d":1004}
s = pd.Series(data)
print(s)
print(s[1])
print(s[2:3])
```

```
a    1001
b    1002
c    1003
d    1004
dtype: int64
1002
c    1003
dtype: int64
/tmp/ipykernel_6416/729615010.py:5: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
print(s[1])
```

```
#employee salaries
sal = [35000,42000,38000,50000,45000]
s=pd.Series(sal,index = [1001,1002,1003,1004,1005])
print(s)
#incre sal 1003 to 40000
s[1003] = 40000
print(s)
#delete emp 1002
s=s.drop(1002)
print(s)
#add emp 1006 with sal 55000
new_data = pd.Series([55000], index=[1006])
s=pd.concat([s,new_data])
print(s)
#dis first 3 emp
print(s[0:3])
print(s)
```

```
1001    35000
1002    42000
1003    38000
1004    50000
1005    45000
dtype: int64
1001    35000
1002    42000
1003    40000
1004    50000
1005    45000
dtype: int64
1001    35000
1003    40000
1004    50000
1005    45000
dtype: int64
1001    35000
1003    40000
1004    50000
1005    45000
1006    55000
dtype: int64
1001    35000
1003    40000
1004    50000
dtype: int64
1001    35000
1003    40000
1004    50000
1005    45000
1006    55000
dtype: int64
```

```
#product prices
#create series
price = [500,750,1200,980,1500]
s=pd.Series(price,index=["p101","p102","p103","p104","p105"])
print(s)
#dis price of p103
print(s["p103"])
#chang price p102 - 800
s["p102"] = 800
print(s)
#delete p104
```

```
s=s.drop("p104")
print(s)
#Add a new product p106 - 2000
new = pd.Series([2000], index=["p106"])
s=pd.concat([s,new])
print(s)
```

```
p101    500
p102    750
p103    1200
p104    980
p105    1500
dtype: int64
1200
p101    500
p102    800
p103    1200
p104    980
p105    1500
dtype: int64
p101    500
p102    800
p103    1200
p105    1500
dtype: int64
p101    500
p102    800
p103    1200
p105    1500
p106    2000
dtype: int64
```

```
#daily temp
#create panda series
temp = [31,32,34,33,35,36,34]
s=pd.Series(temp, index = ["monday","tuesday","wed","thur","friday","sat","sun"])
print(s)
#display temp from mon to wed
print(s["monday":"wed"])
#change thur temp to 32
s["thur"] = 32
print(s)
#delete sat temp
s=s.drop("sat")
print(s)
#add mon of the next week with temp 30
new = pd.Series([30],index = ["monday"])
s = pd.concat([s,new])
print(s)
```

```
monday    31
tuesday   32
wed        34
thur       33
friday     35
sat        36
sun        34
dtype: int64
monday    31
tuesday   32
wed        34
dtype: int64
monday    31
tuesday   32
wed        34
thur       32
friday     35
sat        36
sun        34
dtype: int64
monday    31
tuesday   32
wed        34
thur       32
friday     35
sat        36
sun        34
dtype: int64
monday    31
tuesday   32
wed        34
thur       32
friday     35
sat        36
sun        34
```

```
monday    30
dtype: int64
```

```
#bank acc
#cre series
bal = [25000,45000,32000,18000,60000]
s = pd.Series(bal,index = ["a101","a102","a103","a104","a105"])
print(s)
#dis the bal amt a103
print(s["a103"])
#update a103 - 25000
s["a103"] = 25000
print(s)
#delete acc a101
s=s.drop("a101")
print(s)
#add acc a106 with bal 40000
new = pd.Series([40000],index = ["a106"])
s=pd.concat([s,new])
print(s)
```

```
a101    25000
a102    45000
a103    32000
a104    18000
a105    60000
dtype: int64
32000
a101    25000
a102    45000
a103    25000
a104    18000
a105    60000
dtype: int64
a102    45000
a103    25000
a104    18000
a105    60000
dtype: int64
a102    45000
a103    25000
a104    18000
a105    60000
a106    40000
dtype: int64
```

```
import pandas as pd
data = [5000,7000,6000,9000,8000]
s=pd.Series(data,index = ["monday","tuesday","wednesday","thursday","friday"])
print(s)
#viewing
# head for viewing top rows
print(s.head(2))

#tail for last rows
print(s.tail(2))

#length
print(len(s))

#dimension
print(s.shape)

#index
print(s.index)

#to show the values
print(s.values)
```

```
monday    5000
tuesday    7000
wednesday    6000
thursday    9000
friday    8000
dtype: int64
monday    5000
tuesday    7000
dtype: int64
thursday    9000
friday    8000
dtype: int64
```

```
5
(5,)
Index(['monday', 'tuesday', 'wednesday', 'thursday', 'friday'], dtype='object')
[5000 7000 6000 9000 8000]
```

```
import pandas as pd
data = [5000,7000,6000,9000,8000]
s=pd.Series(data,index = ["monday","tuesday","wednesday","thursday","friday"])
print(s)
#filtering
#select using single index
print(s["monday"])

#mutliple values
print(s[["monday","friday"]])

#iloc[]-it gives index values,integer based
print(s.iloc[1])
print(s.iloc[4])

#select a range
print(s.iloc[0:3])

#filtering using a condtn
print(s>6000)
print(s[s>6000])

print(s[s<6000])
```

```
monday      5000
tuesday     7000
wednesday   6000
thursday    9000
friday      8000
dtype: int64
5000
monday      5000
friday     8000
dtype: int64
7000
8000
monday      5000
tuesday     7000
wednesday   6000
dtype: int64
monday      False
tuesday     True
wednesday   False
thursday    True
friday      True
dtype: bool
tuesday     7000
thursday    9000
friday      8000
dtype: int64
monday      5000
dtype: int64
```

```
#cleaning data
import pandas as pd
data = [5000,None,6000,None,8000]
s=pd.Series(data,index = [1,2,3,4,5])
print(s)

#identify missing values
print(s.isna())

#countng missing values - s.isna().sum()
print(s.isna().sum())

#to drop (removing)
#print(s.dropna())

#to prmntly update
#s=s.dropna()
#print(s)

#to fill the data insted of deleting in the place of none values
print(s.fillna(2))

#use to fill the value with avg value
print(s.fillna(s.mean()))
```

```

1    5000.0
2      NaN
3    6000.0
4      NaN
5    8000.0
dtype: float64
1    False
2     True
3    False
4     True
5    False
dtype: bool
2
1    5000.0
2      2.0
3    6000.0
4      2.0
5    8000.0
dtype: float64
1    5000.000000
2    6333.333333
3    6000.000000
4    6333.333333
5    8000.000000
dtype: float64

```

```

#create a series of a wheater report in 2025 - 12 months jan-20, feb-25, march-none, april-30,may-40,june-none,july-none,aug-20,
w = [20,25,None,30,40,None,None,36,22,20,None,10]
s=pd.Series(w,index=["jan","feb","march","apr","may","june","july","aug","sept","oct","nov","dec"])
print(s)
#viweing
print(s.head(3))
print(s.tail(2))
print(len(s))
print(s.shape)
print(s.index)
print(s.values)

#filtering
print(s["may"])
print(s[["jan","may"]])
print(s.iloc[3])
print(s.iloc[1:3])
print(s>30)

#cleaning data
print(s.isna())
print(s.isna().sum())
print(s.fillna(s.mean()))

#aggreation
#sum-total
print(s.sum())
#mean - avg
print(s.mean())
#min - lowest
print(s.min())
#max- highest
print(s.max())
#count
print(s.count())
#median - middle term
print(s.median())
#standard deviation
print(s.std())

```

```

jan      20.0
feb      25.0
march    NaN
apr      30.0
may      40.0
june     NaN
july     NaN
aug      36.0
sept     22.0
oct      20.0
nov      NaN
dec      10.0
dtype: float64
jan      20.0
feb      25.0

```

```

march    NaN
dtype: float64
nov      NaN
dec     10.0
dtype: float64
12
(12,)
Index(['jan', 'feb', 'march', 'apr', 'may', 'june', 'july', 'aug', 'sept',
      'oct', 'nov', 'dec'],
      dtype='object')
[20. 25. nan 30. 40. nan nan 36. 22. 20. nan 10.]
40.0
jan     20.0
may     40.0
dtype: float64
30.0
feb      25.0
march    NaN
dtype: float64
jan      False
feb      False
march    False
apr      False
may       True
june     False
july     False
aug       True
sept     False
oct      False
nov      False
dec      False
dtype: bool
jan      False
feb      False
march     True
apr      False
may      False
june     True
july     True
aug      False
sept     False
oct      False

```

```

#group by - used in multiple column we use dataframe- having same values will be grouped
#merging/joining - use pd.concat
#joining-used in series
s1 = pd.Series([1,2,3,4])
s2 = pd.Series([5,6,7,8])
res = pd.concat([s1,s2],ignore_index=True)#index wont repeat
print(res)

#merge- we can only use concat, used in dataframe

```

```

0    1
1    2
2    3
3    4
4    5
5    6
6    7
7    8
dtype: int64

```

```

#DataFrame- a DF is two-dimensional labeled data structure in pandas -think it as an excel table or sql table
#create a dataframe
import pandas as pd #list
s = [
    #nested list - list inside another list
    ["Ravi",22,30000],
    ["Priya",24,40000],
    ["Arun",21,28000]
]
df = pd.DataFrame(s,columns=["name","age","salary"])
print(df)

```

```

   name  age  salary
0  Ravi   22   30000
1  Priya  24   40000
2  Arun   21   28000

```



```
import pandas as pd #tuple
s=[("aa",23,700),("bb",32,500),("cc",43,900)]
n = pd.DataFrame(s,columns=["name","age","salary"])
print(n)
```

```
   name  age  salary
0    aa   23     700
1    bb   32     500
2    cc   43     900
```

```
#E-commerce sales analysis
import pandas as pd
sales = [25000,28000,32000,None,35000,38000,None,42000,45000,48000]
#Create the Series with dates as indexes.
s=pd.Series(sales,index = ["aug-1","aug-2","aug-3","aug-4","aug-5","aug-6","aug-7","aug-8","aug-9","aug-10"])
print(s)

#Display the first 5 days.
print(s.head(5))

#Display the last 3 days.
print(s.tail(3))

#Find all missing values.
print(s.isna())

#Count the missing values.
print(s.isna().sum())

#Replace missing values with the average sales.
print(s.fillna(s.mean()))

#Change the sales of 03-Aug to ₹34,000.
s["aug-3"] = 34000
print(s)

#Delete the sales data for 04-Aug.
print(s.drop("aug-4"))

#Add sales for 11-Aug as ₹50,000.
new_data = pd.Series([50000],index = ["aug-11"])
n = pd.concat([s,new_data])
print(n)

#Find total sales.
print(s.sum())

#Find average sales.
print(s.mean())

#Find average sales.
print(s.max())

#Find the lowest sales.
print(s.min())
```

```
aug-1    25000.0
aug-2    28000.0
aug-3    32000.0
aug-4         NaN
aug-5    35000.0
aug-6    38000.0
aug-7         NaN
aug-8    42000.0
aug-9    45000.0
aug-10   48000.0
dtype: float64
aug-1    25000.0
aug-2    28000.0
aug-3    32000.0
aug-4         NaN
aug-5    35000.0
dtype: float64
aug-8    42000.0
aug-9    45000.0
aug-10   48000.0
```

```

dtype: float64
aug-1      False
aug-2      False
aug-3      False
aug-4       True
aug-5      False
aug-6      False
aug-7       True
aug-8      False
aug-9      False
aug-10     False
dtype: bool
2
aug-1      25000.0
aug-2      28000.0
aug-3      32000.0
aug-4      36625.0
aug-5      35000.0
aug-6      38000.0
aug-7      36625.0
aug-8      42000.0
aug-9      45000.0
aug-10     48000.0
dtype: float64
aug-1      25000.0
aug-2      28000.0
aug-3      34000.0
aug-4      NaN
aug-5      35000.0
aug-6      38000.0
aug-7      NaN
aug-8      42000.0
aug-9      45000.0
aug-10     48000.0
dtype: float64
aug-1      25000.0
aug-2      28000.0
aug-3      34000.0

```

```

#Complete Data Analyst Task- A retail store gives you the following monthly revenue data:
#Creating
#Create a Pandas Series with Month as the index.
rev = [120000,135000,None,150000,165000,None,180000,195000,210000,225000]
n = pd.Series(rev,index=["jan","feb","march","april","may","june","july","aug","sept","oct"])
print(n)

#Viewing
#2. Display the first 5 months.
print(n.head(5))
#3. Display the last 3 months.
print(n.tail(3))

#Selecting
#4. Display January, March and May.
print(n[["jan","march","april"]])

#5. Display revenue greater than ₹170,000.
print(n[n>170000])

#Cleaning
#6. Identify missing values.
print(n.isna())

#7. Count missing values.
print(n.isna().sum())

#8. Fill missing values using the average revenue.
print(n.fillna(n.mean()))

#Manipulating
#Change April revenue to ₹155,000.
n["april"] = 155000
print(n)

#Delete February.
print(n.drop("feb"))

#Add November with revenue ₹240,000.
new = pd.Series([240000],index=["nov"])
g = pd.concat([n,new])
print(g)

#Aggregation
# Calculate total revenue
print(n.sum())
#Calculate average revenue.

```

```

print(n.mean())
# Find maximum revenue.
print(n.max())
# Find minimum revenue.
print(n.min())

#Combining
#Create another Series for December and combine it with the existing Series using pd.concat().
rev = 5000
new_data = pd.Series(rev,index = ["dec"])
k = pd.concat([n,new_data])
print(k)

```

```

jan      120000.0
feb      135000.0
march      NaN
april     150000.0
may       165000.0
june      172500.0
july      180000.0
aug       195000.0
sept      210000.0
oct       225000.0
dtype: float64
jan      120000.0
feb      135000.0
march      NaN
april     150000.0
may       165000.0
dtype: float64
aug       195000.0
sept      210000.0
oct       225000.0
dtype: float64
jan      120000.0
march      NaN
april     150000.0
dtype: float64
july      180000.0
aug       195000.0
sept      210000.0
oct       225000.0
dtype: float64
jan      False
feb      False
march     True
april     False
may      False
june     True
july     False
aug      False
sept     False
oct      False
dtype: bool
2
jan      120000.0
feb      135000.0
march     172500.0
april     150000.0
may       165000.0
june     172500.0
july      180000.0
aug       195000.0
sept      210000.0
oct       225000.0
dtype: float64
jan      120000.0
feb      135000.0
march      NaN
april     155000.0
may       165000.0

```

```

import pandas as pd
s = {1,2,3,4,5,6}
d=pd.DataFrame(list(s)) #list is used bcz set is unordered so it cnvrt into lis
print(d)

```

```

0
0 1
1 2
2 3
3 4
4 5
5 6

```

```

#DataFrame
import pandas as pd
data = {
    "Name": ["Girisha", "Neha", "Jyothi", "ashwini", "chandini"],
    "Age": [21, 22, 23, 24, 25],
    "City": ["vizag", "hyd", "bnglr", "chennai", "mumbai"]
}
df = pd.DataFrame(data)
print(df)

#viewing
print(df.head(2))
print(df.tail(3))
print(len(df))
print(df.shape)
print(df.shape[1])
print(df.index)
print(df.values)
print(df.info())      #DataFrame method which is not used in Series
print(df.describe())

#selecting
print(df['Name'])
print(df[['Name', 'Age']])
print(df.iloc[0])
print(df.iloc[0:3])
print(df.iloc[1,2])  #it gives output for 1st row 1st column value
print(df.iloc[0,1])  # 0 index in 1st row
#if we give value then it should print its index postiton
print

#to get a specifc column with a specific condition

print(df[df['Age'] > 22]) #if we use like this than we get the values

print(df['Age'] > 23) #if we use like this than we get true or false(boolean)

#multiple conditions AND- both conditions shld be true
res = df[
    (df['Age'] > 21) &
    (df['City'] == 'hyd')
]
print(res)

```

```

      Name  Age  City
0  Girisha  21  vizag
1    Neha  22   hyd
2  Jyothi  23  bnglr
3  ashwini  24  chennai
4  chandini  25  mumbai
      Name  Age  City
0  Girisha  21  vizag
1    Neha  22   hyd
      Name  Age  City
2  Jyothi  23  bnglr
3  ashwini  24  chennai
4  chandini  25  mumbai
5
(5, 3)
3
RangeIndex(start=0, stop=5, step=1)
[['Girisha' 21 'vizag']
 ['Neha' 22 'hyd']
 ['Jyothi' 23 'bnglr']
 ['ashwini' 24 'chennai']
 ['chandini' 25 'mumbai']]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0    Name    5 non-null    object
1    Age      5 non-null    int64
2    City     5 non-null    object
dtypes: int64(1), object(2)
memory usage: 252.0+ bytes
None
      Age
count  5.000000
mean   23.000000
std     1.581139
min     21.000000
25%     22.000000

```

```

50%    23.000000
75%    24.000000
max     25.000000
0      Girisha
1        Neha
2       Jyothi
3     ashwini
4    chandini
Name: Name, dtype: object
      Name  Age
0  Girisha   21
1   Neha    22
2  Jyothi   23
3  ashwini   24
4  chandini   25
Name  Girisha
Age    21
City   vizag
Name: 0, dtype: object

```

```

import pandas as pd
data = {
    "Name": ["Girisha", "Neha", "Jyothi", "ashwini", "chandini"],
    "Age": [21, 22, 23, 24, 25],
    "City": ["vizag", "hyd", "bnglr", "chennai", "mumbai"]
}
df = pd.DataFrame(data)
print(df)

```

```

import pandas as pd
data = {
    "Name": ["Girisha", "Neha", "Jyothi", None, None],
    "Age": [21, 22, 23, None, 25],
    "City": ["vizag", "hyd", "bnglr", None, "mumbai"]
}
df = pd.DataFrame(data)
print(df)

#cleaning
print(df.isna()) #print true or false
print(df.isna().sum()) #counting missing values
print(df.fillna(df.mean())) #fill with avg values

```

```

      Name  Age  City
0  Girisha  21.0  vizag
1   Neha   22.0   hyd
2  Jyothi  23.0  bnglr
3     None   NaN   None
4     None  25.0  mumbai
      Name  Age  City
0  False  False  False
1  False  False  False
2  False  False  False
3   True   True   True
4   True  False  False
Name      2
Age       1
City      1
dtype: int64

```

```

-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_1388/1806643988.py in <cell line: 0>()
    12 print(df.isna()) #print true or false
    13 print(df.isna().sum()) #counting missing values
--> 14 print(df.fillna(df.mean())) #fill with avg values

```

↕ 10 frames

```

/usr/local/lib/python3.12/dist-packages/numpy/_core/_methods.py in _sum(a, axis, dtype, out, keepdims, initial, where)
    50 def _sum(a, axis=None, dtype=None, out=None, keepdims=False,
    51          initial=_NoValue, where=True):
--> 52     return umr_sum(a, axis, dtype, out, keepdims, initial, where)
    53
    54 def _prod(a, axis=None, dtype=None, out=None, keepdims=False,

```

```

TypeError: can only concatenate str (not "int") to str

```

```

s1 = [0,2,4,6,8]
s=pd.Series(s1)
s2 = [1,3,5,7,9]
n = pd.Series(s2)

```

```

s3 = [2,3,5,7,11,13]
m=pd.Series(s3)
d1 = s.to_frame()
print(d1)
d2 = n.to_frame()
print(d2)
d3 = m.to_frame()
print(d3)
data = {
    "num-1" : s,
    "num-2" : n,
    "num-3" : m
}
b = pd.DataFrame(data)
print(b)

print(b.iloc[2,2])

```

```

0
0 0
1 2
2 4
3 6
4 8
0
0 1
1 3
2 5
3 7
4 9
0
0 2
1 3
2 5
3 7
4 11
5 13
num-1  num-2  num-3
0    0.0    1.0     2
1    2.0    3.0     3
2    4.0    5.0     5
3    6.0    7.0     7
4    8.0    9.0    11
5    NaN    NaN    13
5

```

```

import pandas as pd
data = {
    "Name": ["Girisha","Neha","Jyothi","ashwini","chandini"],
    "Age": [21,22,23,24,25],
    "City": ["vizag","hyd","bnglr","chennai","mumbai"]
}
df = pd.DataFrame(data)
print(df)
#rename column names into a,b,c
df.columns = ['A','B','C']  #multiple columns
print(df)

```

```

      Name  Age  City
0  Girisha  21  vizag
1   Neha   22   hyd
2  Jyothi  23  bnglr
3 ashwini  24  chennai
4 chandini  25  mumbai
      A    B    C
0  Girisha  21  vizag
1   Neha   22   hyd
2  Jyothi  23  bnglr
3 ashwini  24  chennai
4 chandini  25  mumbai

```

```
df.rename(
    columns={
        'A' : "Alpha",
        'B' : "Beta",
        'C' : "gamma"
    },
    inplace=True
)
print(df)
```

	Alpha	Beta	gamma
0	Girisha	21	vizag
1	Neha	22	hyd
2	Jyothi	23	bnglr
3	ashwini	24	chennai
4	chandini	25	mumbai

```
import pandas as pd
data = {
    "Name" : ["Ravi", "Priya", "arun", "sneha", "kiran", "anjali", "vikram", "meena"],
    "Age" : [22, 24, 21, 25, 23, 26, 20, 27],
    "City" : ["hyd", "chennai", "bnglr", "hyd", "mumbai", "chennai", "bnglr", "mumbai"],
    "Salary" : [30000, 40000, 28000, 50000, 35000, 45000, 25000, 55000]
}
df = pd.DataFrame(data)
print(df)
print(df['Name'])
print(df[['Name', 'Salary']])
print(df[df['Age'] > 23])
print(df[df['Salary'] > 40000])
print(df[df["City"] == 'chennai'])
res = df[
    (df['Age'] > 22) &
    (df['Salary'] > 35000)
]
print(res)
re = df[
    (df['City'] == "mumbai") |
    (df['City'] == "bnglr")
]
print(re)
print(df[df['Salary'] > 30000][['Name', 'Salary']])
print(df.iloc[2:5])
print(df.head(4)[['Name', 'Age']])
print(df.iloc[[0, 3, 6]])
print(df.iloc[2:6])
df.rename(columns={
    "Name": "Employee_Name",
    "Age": "Employee_Age",
    "City": "Location",
    "Salary": "Monthly_Salary"
}, inplace=True)

print(df)
print(df[(df["Employee_Age"] >= 22) &
    (df["Employee_Age"] <= 26)]
    [["Employee_Name", "Location", "Monthly_Salary"]])
```

	Name	Age	City	Salary
0	Ravi	22	hyd	30000
1	Priya	24	chennai	40000
2	arun	21	bnglr	28000
3	sneha	25	hyd	50000
4	kiran	23	mumbai	35000
5	anjali	26	chennai	45000
6	vikram	20	bnglr	25000
7	meena	27	mumbai	55000

0 Ravi
1 Priya
2 arun
3 sneha
4 kiran
5 anjali
6 vikram
7 meena

Name: Name, dtype: object

	Name	Salary
0	Ravi	30000
1	Priya	40000
2	arun	28000
3	sneha	50000
4	kiran	35000

```

5  anjali  45000
6  vikram  25000
7  meena  55000
   Name Age City Salary
1  Priya  24 chennai 40000
3  sneha  25 hyd 50000
5  anjali  26 chennai 45000
7  meena  27 mumbai 55000
   Name Age City Salary
3  sneha  25 hyd 50000
5  anjali  26 chennai 45000
7  meena  27 mumbai 55000
   Name Age City Salary
1  Priya  24 chennai 40000
5  anjali  26 chennai 45000
   Name Age City Salary
1  Priya  24 chennai 40000
3  sneha  25 hyd 50000
5  anjali  26 chennai 45000
7  meena  27 mumbai 55000
   Name Age City Salary
2  arun  21 bnglr 28000
4  kiran  23 mumbai 35000
6  vikram  20 bnglr 25000
7  meena  27 mumbai 55000
   Name Age City Salary
1  Priya  24 chennai 40000
3  sneha  25 hyd 50000
4  kiran  23 mumbai 35000
5  anjali  26 chennai 45000
7  meena  27 mumbai 55000 [['Name', 'Salary']]
   Name Age City Salary
2  arun  21 bnglr 28000

```

```

#day-5
#cleaning of data in dataframe
#create a dataframe of a week sales profit,loss,discount
import pandas as pd
data = {
    "Week" : ["mon", "tue", "wed", "thur", "frdy", "sat", "sun"],
    "Sales" : [5000, None, None, 3000, None, 3000, 2000],
    "Profit" : [50, None, None, 40, None, 59, 76],
    "Loss" : [10, None, None, 29, None, 70, 50],
    "Discount" : [5, 32, 43, 23, 54, 50, 45]
}
df = pd.DataFrame(data)
print(df)
print(df.isnull())
print(df.dropna())
print(df.dropna(axis=1))#it display columns which doesnt contain missing value
#axis is used bcs it contains multiple col to display particular col we use axis 1-col,0-row
print(df.drop("Loss",axis=1))
print(df.drop("Sales",axis=1))
print(df.drop(1,axis=0)) #del a particular row
print(df.dropna(inplace=True)) #to remove permntly missing values

```

```

   Week Sales Profit Loss Discount
0  mon  5000.0   50.0  10.0        5
1  tue    NaN    NaN   NaN        32
2  wed    NaN    NaN   NaN        43
3  thur  3000.0   40.0  29.0        23
4  frdy    NaN    NaN   NaN        54
5  sat  3000.0   59.0  70.0        50
6  sun  2000.0   76.0  50.0        45
   Week Sales Profit Loss Discount
0  False False False False False
1  False True True True False
2  False True True True False
3  False False False False False
4  False True True True False
5  False False False False False
6  False False False False False
   Week Sales Profit Loss Discount
0  mon  5000.0   50.0  10.0        5
3  thur  3000.0   40.0  29.0        23
5  sat  3000.0   59.0  70.0        50
6  sun  2000.0   76.0  50.0        45
   Week Discount
0  mon        5
1  tue       32
2  wed       43
3  thur       23
4  frdy       54
5  sat       50
6  sun       45

```


	Week	Sales	Profit	Discount
0	mon	5000.0	50.0	5
1	tue	NaN	NaN	32
2	wed	NaN	NaN	43
3	thur	3000.0	40.0	23
4	frdy	NaN	NaN	54
5	sat	3000.0	59.0	50
6	sun	2000.0	76.0	45

	Week	Profit	Loss	Discount
0	mon	50.0	10.0	5
1	tue	NaN	NaN	32
2	wed	NaN	NaN	43
3	thur	40.0	29.0	23
4	frdy	NaN	NaN	54
5	sat	59.0	70.0	50
6	sun	76.0	50.0	45

	Week	Sales	Profit	Loss	Discount
0	mon	5000.0	50.0	10.0	5
2	wed	NaN	NaN	NaN	43
3	thur	3000.0	40.0	29.0	23
4	frdy	NaN	NaN	NaN	54
5	sat	3000.0	59.0	70.0	50
6	sun	2000.0	76.0	50.0	45

None

```
import pandas as pd
data = {
    "Week" : ["mon","tue","wed","thur","frdy","sat","sun"],
    "Sales" : [5000,None,None,3000,None,3000,2000],
    "Profit" : [50,None,None,40,None,59,76],
    "Loss" : [10,None,None,29,None,70,50],
    "Discount" : [5,32,43,23,54,50,45]
}
df = pd.DataFrame(data)
print(df)
#filling missing values
print(df.fillna(3)) #fill missing values with 3
rem = df.fillna({ #fill null values with particular given condition sales - in null - fill with 2
    'Week' : 1,
    'Sales' : 2,
    'Profit' : 3,
    'Loss' : 4,
    'Discount' : 5
})

print(rem)

#forward filling - df.ffmpeg() - we can fill with old data
print(df.ffmpeg())

#to fill particular col with mean,median,stdv values
m= df['Sales'] = df['Sales'].fillna(
    df['Sales'].mean()
)
print(m)

k = df['Profit'] = df['Profit'].fillna(
    df['Profit'].mean()
)
print(k)

h = df['Loss'] = df['Loss'].fillna(
    df['Loss'].mean()
)
print(h)
```

	Week	Sales	Profit	Loss	Discount
0	mon	5000.0	50.0	10.0	5
1	tue	NaN	NaN	NaN	32
2	wed	NaN	NaN	NaN	43
3	thur	3000.0	40.0	29.0	23
4	frdy	NaN	NaN	NaN	54
5	sat	3000.0	59.0	70.0	50
6	sun	2000.0	76.0	50.0	45

	Week	Sales	Profit	Loss	Discount
0	mon	5000.0	50.0	10.0	5
1	tue	3.0	3.0	3.0	32
2	wed	3.0	3.0	3.0	43
3	thur	3000.0	40.0	29.0	23
4	frdy	3.0	3.0	3.0	54
5	sat	3000.0	59.0	70.0	50
6	sun	2000.0	76.0	50.0	45

```

Week  Sales  Profit  Loss  Discount
0  mon  5000.0   50.0  10.0         5
1  tue    2.0    3.0   4.0        32
2  wed    2.0    3.0   4.0        43
3  thur  3000.0   40.0  29.0        23
4  frdy    2.0    3.0   4.0        54
5  sat  3000.0   59.0  70.0        50
6  sun  2000.0   76.0  50.0        45
Week  Sales  Profit  Loss  Discount
0  mon  5000.0   50.0  10.0         5
1  tue  5000.0   50.0  10.0        32
2  wed  5000.0   50.0  10.0        43
3  thur  3000.0   40.0  29.0        23
4  frdy  3000.0   40.0  29.0        54
5  sat  3000.0   59.0  70.0        50
6  sun  2000.0   76.0  50.0        45
0    5000.0
1    3250.0
2    3250.0
3    3000.0
4    3250.0
5    3000.0
6    2000.0
Name: Sales, dtype: float64
0    50.00
1    56.25
2    56.25
3    40.00
4    56.25
5    59.00
6    76.00
Name: Profit, dtype: float64
0    10.00
1    39.75
2    39.75
3    29.00
4    39.75
5    70.00
6    50.00
Name: Loss, dtype: float64

```

```

#manuplating dataset
import pandas as pd
data = {
    "Week" : ["mon","tue","wed","thur","frdy","sat","sun"],
    "Sales" : [500,None,None,3000,None,3000,2000],
    "Profit" : [5000,None,None,40,None,59,76],
    "Loss" : [10,None,None,29,None,70,50],
    "Discount" : [5,32,43,23,54,50,45]
}
df = pd.DataFrame(data)
print(df)

#to add new col
g = df['ID'] = [101,102,103,104,105,106,107]
print(g)

#creating an new col with some cal value
k = df['Bonus'] = df['Sales'] * 0.5
print(k)

k = df['Discount'] = df['Discount'] + 100
print(k)

#to change a specific value
df.loc[
    df['Week'] == 'mon', 'Profit'] = 50000
print(df)

#len
df.loc[len(df)] = ['rainbow',40,23,6,54,108,90]
print(df)

```

```

Week  Sales  Profit  Loss  Discount
0  mon  500.0  5000.0  10.0         5
1  tue    NaN    NaN   NaN        32
2  wed    NaN    NaN   NaN        43
3  thur  3000.0   40.0  29.0        23
4  frdy    NaN    NaN   NaN        54
5  sat  3000.0   59.0  70.0        50
6  sun  2000.0   76.0  50.0        45
[101, 102, 103, 104, 105, 106, 107]
0    250.0
1      NaN
2      NaN
3    1500.0

```

```

4      NaN
5    1500.0
6    1000.0
Name: Sales, dtype: float64
0    105
1    132
2    143
3    123
4    154
5    150
6    145
Name: Discount, dtype: int64
   Week  Sales  Profit  Loss  Discount  ID  Bonus
0  mon   500.0  50000.0  10.0        105  101  250.0
1  tue     NaN     NaN   NaN         132  102     NaN
2  wed     NaN     NaN   NaN         143  103     NaN
3  thur  3000.0    40.0  29.0         123  104  1500.0
4  frdy     NaN     NaN   NaN         154  105     NaN
5  sat  3000.0    59.0  70.0         150  106  1500.0
6  sun  2000.0    76.0  50.0         145  107  1000.0
   Week  Sales  Profit  Loss  Discount  ID  Bonus
0  mon   500.0  50000.0  10.0        105  101  250.0
1  tue     NaN     NaN   NaN         132  102     NaN
2  wed     NaN     NaN   NaN         143  103     NaN
3  thur  3000.0    40.0  29.0         123  104  1500.0
4  frdy     NaN     NaN   NaN         154  105     NaN
5  sat  3000.0    59.0  70.0         150  106  1500.0
6  sun  2000.0    76.0  50.0         145  107  1000.0
7 rainbow   40.0    23.0   6.0          54  108    90.0

```

```

import pandas as pd

# Creating DataFrame
data = {
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran"],
    "Age": [22, 24, 21, 25, 23],
    "City": ["Hyderabad", "Chennai", "Bangalore", "Mumbai", "Delhi"],
    "Salary": [35000, 45000, 30000, 50000, 40000]
}

df = pd.DataFrame(data)

print("Original DataFrame")
print(df)

```

```

Original DataFrame
   Name  Age  City  Salary
0  Ravi   22  Hyderabad  35000
1  Priya  24   Chennai  45000
2  Arun   21  Bangalore  30000
3  Sneha  25   Mumbai  50000
4  Kiran  23    Delhi  40000

```

```

#merging DataFrames - kind of an sql joins
#how - parameter determines which matching and non matching records are retained

import pandas as pd
df1 = pd.DataFrame({
    'ID' : [1,2,3],
    'Name' : ["pinky","sweety","honey"]
})

df2 = pd.DataFrame({
    'ID' : [1,2,4],
    'Salary' : [2000,3000,5000]
})
print(df1)
print(df2)

#inner join
m = pd.merge(
    df1,
    df2,
    on = 'ID',
    how = 'inner'
)
print(m)

#left join
k = pd.merge(
    df1,
    df2,
    on = 'ID',
    how = 'left'
)

```

```

)
print(k)

#right join
l = pd.merge(
    df1,
    df2,
    on = 'ID',
    how = 'right'
)
print(l)

#outer join
n = pd.merge(
    df1,
    df2,
    on = 'ID',
    how = 'outer'
)
print(m)

```

```

ID    Name
0  1  pinky
1  2  sweety
2  3  honey
ID    Salary
0  1    2000
1  2    3000
2  4    5000
ID    Name  Salary
0  1  pinky    2000
1  2  sweety    3000
ID    Name  Salary
0  1  pinky  2000.0
1  2  sweety  3000.0
2  3  honey   NaN
ID    Name  Salary
0  1  pinky    2000
1  2  sweety    3000
2  4   NaN     5000
ID    Name  Salary
0  1  pinky    2000
1  2  sweety    3000

```

#pd.concat - stackData - it doesnt need common col
 #concat - helps to combines dataframes along rows and col,combine use axis = 0 - row ,axis = 1- col
 #if we keep ignore_index - then it will avoid it , doesnt allow duplicate index

```

import pandas as pd
df1 = pd.DataFrame({
    'ID' : [1,2,3],
    'Name' : ["pink", "sweet", "honey"]
})

df2 = pd.DataFrame({
    'ID' : [1,2,4],
    'Salary' : [2000,3000,5000]
})
print(df1)
print(df2)
#for row
c = pd.concat(
    [df1,df2],
    axis= 0,
    ignore_index= True #if we wont use this then we get repeated index when we combine col
)
print(c)
#for col
d = pd.concat(
    [df1,df2],
    axis= 1,
    ignore_index= True #if we wont use this then we get repeated index when we combine col
)
print(d)

```

```

ID    Name
0  1  pinky
1  2  sweety
2  3  honey
ID    Salary
0  1    2000
1  2    3000
2  4    5000
ID    Name  Salary
0  1  pinky   NaN

```

```

1 2 2 1 2 3
2 3 1 2 3
3 1 2 3
4 2 3 4
5 4 5 6
0 1 2 3
0 1 2 3
1 2 3 4
2 3 4 5

```

```

#CSV (commoa-separated vaules) - Read & Write
import pandas as pd
df = pd.read_csv("match.csv")
print(df)
print(df.head())

```

```

match_id world_cup_year host_country date \
0 1 1930 Uruguay 1930-07-13
1 2 1930 Uruguay 1930-07-15
2 3 1930 Uruguay 1930-07-16
3 4 1930 Uruguay 1930-07-19
4 5 1930 Uruguay 1930-07-19
... ..
1064 1065 2026 Canada, Mexico, United States 2026-07-11
1065 1066 2026 Canada, Mexico, United States 2026-07-14
1066 1067 2026 Canada, Mexico, United States 2026-07-15
1067 1068 2026 Canada, Mexico, United States 2026-07-18
1068 1069 2026 Canada, Mexico, United States 2026-07-19

```

```

stage round_raw group team1 \
0 Group Stage Matchday 1 Group 1 France
1 Group Stage Matchday 3 Group 1 Argentina
2 Group Stage Matchday 4 Group 1 Chile
3 Group Stage Matchday 7 Group 1 Chile
4 Group Stage Matchday 7 Group 1 Argentina
... ..
1064 Quarter-final Quarter-finals NaN Argentina
1065 Semi-final Semi-finals NaN France
1066 Semi-final Semi-finals NaN England
1067 Third Place Play-off Match for third place NaN France
1068 Final Final NaN Spain

```

```

team2 halftime_score_team1 ... extra_time_score_team1 \
0 Mexico 3.0 ... NaN
1 France 0.0 ... NaN
2 Mexico 1.0 ... NaN
3 France 0.0 ... NaN
4 Mexico 3.0 ... NaN
... ..
1064 Switzerland 1.0 ... 3.0
1065 Spain 0.0 ... NaN
1066 Argentina 0.0 ... NaN
1067 England 0.0 ... NaN
1068 Argentina 0.0 ... 1.0

```

```

extra_time_score_team2 penalty_score_team1 penalty_score_team2 \
0 NaN NaN NaN
1 NaN NaN NaN
2 NaN NaN NaN
3 NaN NaN NaN
4 NaN NaN NaN
... ..
1064 1.0 NaN NaN
1065 NaN NaN NaN
1066 NaN NaN NaN
1067 NaN NaN NaN
1068 0.0 NaN NaN

```

```

winner result_method stadium city \
0 France Normal Time Estadio Pocitos Montevideo
1 Argentina Normal Time Estadio Parque Central Montevideo
2 Chile Normal Time Estadio Parque Central Montevideo
3 Chile Normal Time Estadio Centenario Montevideo
4 Argentina Normal Time Estadio Centenario Montevideo

```

```

#CSV
#to copy path and display
import pandas as pd
df = pd.read_csv("/content/match.csv")
print(df)
print(df.tail())

```

```

match_id world_cup_year host_country date \
0 1 1930 Uruguay 1930-07-13
1 2 1930 Uruguay 1930-07-15
2 3 1930 Uruguay 1930-07-16
3 4 1930 Uruguay 1930-07-19

```

```

4          5          1930          Uruguay  1930-07-19
...      ...      ...      ...      ...
1064      1065      2026  Canada, Mexico, United States  2026-07-11
1065      1066      2026  Canada, Mexico, United States  2026-07-14
1066      1067      2026  Canada, Mexico, United States  2026-07-15
1067      1068      2026  Canada, Mexico, United States  2026-07-18
1068      1069      2026  Canada, Mexico, United States  2026-07-19

      stage      round_raw  group  team1 \
0      Group Stage      Matchday 1  Group 1  France
1      Group Stage      Matchday 3  Group 1  Argentina
2      Group Stage      Matchday 4  Group 1  Chile
3      Group Stage      Matchday 7  Group 1  Chile
4      Group Stage      Matchday 7  Group 1  Argentina
...      ...      ...      ...      ...
1064      Quarter-final      Quarter-finals      NaN  Argentina
1065      Semi-final      Semi-finals      NaN  France
1066      Semi-final      Semi-finals      NaN  England
1067  Third Place Play-off  Match for third place      NaN  France
1068      Final      Final      NaN  Spain

      team2  halftime_score_team1  ...  extra_time_score_team1 \
0      Mexico      3.0  ...      NaN
1      France      0.0  ...      NaN
2      Mexico      1.0  ...      NaN
3      France      0.0  ...      NaN
4      Mexico      3.0  ...      NaN
...      ...      ...      ...      ...
1064  Switzerland      1.0  ...      3.0
1065      Spain      0.0  ...      NaN
1066  Argentina      0.0  ...      NaN
1067  England      0.0  ...      NaN
1068  Argentina      0.0  ...      1.0

      extra_time_score_team2  penalty_score_team1  penalty_score_team2 \
0      NaN      NaN      NaN
1      NaN      NaN      NaN
2      NaN      NaN      NaN
3      NaN      NaN      NaN
4      NaN      NaN      NaN
...      ...      ...      ...
1064      1.0      NaN      NaN
1065      NaN      NaN      NaN
1066      NaN      NaN      NaN
1067      NaN      NaN      NaN
1068      0.0      NaN      NaN

      winner  result_method      stadium  city \
0      France  Normal Time      Estadio Pocitos  Montevideo
1  Argentina  Normal Time  Estadio Parque Central  Montevideo
2      Chile  Normal Time  Estadio Parque Central  Montevideo
3      Chile  Normal Time      Estadio Centenario  Montevideo
4  Argentina  Normal Time      Estadio Centenario  Montevideo

```

```

#json
import pandas as pd
df = pd.read_json("/content/employees.json")
print(df)
print(df.info())
print(df.shape)
print(len(df))

```

```

      id  name  age  department  salary
0      1  Rahul  24          IT  45000
1      2  Priya  26          HR  40000
2      3  Arjun  28      Finance  55000
3      4  Sneha  25          IT  48000
4      5  Vikram  30      Sales  60000
5      6  Anjali  27          HR  42000
6      7  Kiran  29          IT  52000
7      8  Pooja  23      Finance  46000
8      9  Ravi  31      Sales  65000
9     10  Divya  26          IT  50000
10     11  Suresh  32      Finance  70000
11     12  Meena  25          HR  41000
12     13  Naveen  27          IT  53000
13     14  Kavya  24      Sales  47000
14     15  Manoj  30      Finance  62000
15     16  Swathi  28          HR  45000
16     17  Rohit  29          IT  58000
17     18  Lakshmi  26      Sales  51000
18     19  Ajay  33      Finance  75000
19     20  Neha  25          HR  43000
20     21  Vamsi  27          IT  49000
21     22  Aishwarya  24      Sales  46000
22     23  Karthik  31      Finance  68000
23     24  Deepika  29          HR  48000
24     25  Sai  26          IT  54000
25     26  Swathi  28      Sales  59000

```

```

26 27 Tarun 32 Finance 72000
27 28 Harini 23 HR 39000
28 29 Prakash 30 IT 61000
29 30 Bhavya 27 Sales 52000
30 31 Ramesh 34 Finance 78000
31 32 Sowmya 25 HR 44000
32 33 Varun 29 IT 57000
33 34 Keerthi 26 Sales 50000
34 35 Ganesh 31 Finance 69000
35 36 Nandini 24 HR 40000
36 37 Aditya 28 IT 56000
37 38 Madhavi 27 Sales 53000
38 39 Santosh 33 Finance 74000
39 40 Tejaswini 25 HR 45000
40 41 Harsha 30 IT 60000
41 42 Bhanu 26 Sales 49000
42 43 Chaitanya 32 Finance 71000
43 44 Mounika 23 HR 38000
44 45 Surya 29 IT 55000
45 46 Anusha 27 Sales 51000
46 47 Mahesh 35 Finance 80000
47 48 Sravani 24 HR 42000
48 49 Dinesh 31 IT 63000
49 50 Lavanya 26 Sales 54000

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           50 non-null    int64

```

```

#excel
import pandas as pd
df = pd.read_excel("/content/Coca Cola Co.xlsx")
print(df)

```

```

Data provided by SimFin \
0      NaN
1 Profit & Loss statement
2      NaN
3      NaN
4      NaN
..      ...
98     NaN
99     NaN
100    NaN
101    NaN
102    NaN

                                Unnamed: 1 Unnamed: 2 Unnamed: 3 \
0                                NaN          NaN          NaN
1                                NaN          NaN          NaN
2                                in million USD  FY '09    FY '10
3                                NET OPERATING REVENUES  30990    35119
4                                Cost of goods sold    11088    12693
..                                ...          ...          ...
98 Effect of exchange rate changes on cash and ca...    576    -166
99 Cash Provided by (Used in) Investing Activitie...    NaN    NaN
100 Cash Provided by (Used in) Financing Activitie...    NaN    NaN
101 Net Cash Provided by (Used in) Discontinued Op...    NaN    NaN
102 Net increase (decrease) during the year    2320    1496

                                Unnamed: 4 Unnamed: 5 Unnamed: 6 Unnamed: 7 Unnamed: 8 Unnamed: 9 \
0                                NaN          NaN          NaN          NaN          NaN          NaN
1                                NaN          NaN          NaN          NaN          NaN          NaN
2                                FY '11    FY '12    FY '13    FY '14    FY '15    FY '16
3                                46542    48017    46854    45998    44294    41863
4                                18215    19053    18421    17889    17482    16465
..                                ...          ...          ...          ...          ...          ...
98                                -430    -255    -611    -934    -878    -6
99                                NaN          NaN          NaN          NaN          0          0
100                               NaN          NaN          NaN          NaN          0          0
101                               NaN          NaN          NaN          NaN          0          0
102                               4286    -4361    1972    -1456    -1649    1246

                                Unnamed: 10 Unnamed: 11
0                                NaN          NaN
1                                NaN          NaN
2                                FY '17    FY '18
3                                35410    31856
4                                13255    11770
..                                ...          ...
98                                241    -262
99                                -58    -421
100                               -38    205
101                                15    91
102                               -2477    2945

```

```
[103 rows x 12 columns]
```

```
#excel file
#1
import pandas as pd
df = pd.read_excel("/content/Sample - Superstore.xls")
print(df)
#2
print(df.describe())
print(df.info())
#3
print(df.head(10))
#4
print(df.tail(10))
#5
print(df[["Category","Sales"]])
#6
print(df[df["Sales"] > 500])
#7
print(df[df["Profit"] > 100])
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	
...	...	...	...	...	...	...
9989	9990	CA-2014-110422	2014-01-21	2014-01-23	Second Class	
9990	9991	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	
9991	9992	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	
9992	9993	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	
9993	9994	CA-2017-119914	2017-05-04	2017-05-09	Second Class	

	Customer ID	Customer Name	Segment	Country	City	\
0	CG-12520	Claire Gute	Consumer	United States	Henderson	
1	CG-12520	Claire Gute	Consumer	United States	Henderson	
2	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	
3	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	
4	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	
...	...	...	...	...	...	...
9989	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami	
9990	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	
9991	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	
9992	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	
9993	CC-12220	Chris Cortes	Consumer	United States	Westminster	

	...	Postal Code	Region	Product ID	Category	Sub-Category	\
0	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	
1	...	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	...	33311	South	FUR-TA-10000577	Furniture	Tables	
4	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	
...	...	...	...	...	...	...	...
9989	...	33180	South	FUR-FU-10001889	Furniture	Furnishings	
9990	...	92627	West	FUR-FU-10000747	Furniture	Furnishings	
9991	...	92627	West	TEC-PH-10003645	Technology	Phones	
9992	...	92627	West	OFF-PA-10004041	Office Supplies	Paper	
9993	...	92683	West	OFF-AP-10002684	Office Supplies	Appliances	

		Product Name	Sales	Quantity	\
0		Bush Somerset Collection Bookcase	261.9600	2	
1		Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	
2		Self-Adhesive Address Labels for Typewriters b...	14.6200	2	
3		Bretford CR4500 Series Slim Rectangular Table	957.5775	5	
4		Eldon Fold 'N Roll Cart System	22.3680	2	
...		...	...	...	...
9989		Ultra Door Pull Handle	25.2480	3	
9990		Tenex B1-RE Series Chair Mats for Low Pile Car...	91.9600	2	
9991		Aastra 57i VoIP phone	258.5760	2	
9992		It's Hot Message Books with Stickers, 2 3/4" x 5"	29.6000	4	
9993		Acco 7-Outlet Masterpiece Power Center, Wihtou...	243.1600	2	

	Discount	Profit
0	0.00	41.9136
1	0.00	219.5820
2	0.00	6.8714
3	0.45	-383.0310
4	0.20	2.5164

```
#8
print(df[
    (df["Region"] == "West")])
#9
print(df[(df["Quantity"] > 5) & (df["Sales"] > 500)])
#10
```



```
print(df[(df["Region"] == "East") | (df["Region"] == "West")])
#11
print(df.iloc[10:20])
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	
5	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	
6	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	
7	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	
8	9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	
...	...	...	...	...	...	
9986	9987	CA-2016-125794	2016-09-29	2016-10-03	Standard Class	
9990	9991	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	
9991	9992	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	
9992	9993	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	
9993	9994	CA-2017-119914	2017-05-04	2017-05-09	Second Class	

	Customer ID	Customer Name	Segment	Country	City	...	\
2	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	
5	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	
6	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	
7	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	
8	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	
...	...	...	...	...	...	...	
9986	ML-17410	Maris LaWare	Consumer	United States	Los Angeles	...	
9990	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	
9991	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	
9992	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	
9993	CC-12220	Chris Cortes	Consumer	United States	Westminster	...	

	Postal Code	Region	Product ID	Category	Sub-Category	\
2	90036	West	OFF-LA-10000240	Office Supplies	Labels	
5	90032	West	FUR-FU-10001487	Furniture	Furnishings	
6	90032	West	OFF-AR-10002833	Office Supplies	Art	
7	90032	West	TEC-PH-10002275	Technology	Phones	
8	90032	West	OFF-BI-10003910	Office Supplies	Binders	
...	...	...	...	...	...	
9986	90008	West	TEC-AC-10003399	Technology	Accessories	
9990	92627	West	FUR-FU-10000747	Furniture	Furnishings	
9991	92627	West	TEC-PH-10003645	Technology	Phones	
9992	92627	West	OFF-PA-10004041	Office Supplies	Paper	
9993	92683	West	OFF-AP-10002684	Office Supplies	Appliances	

	Product Name	Sales	Quantity	\
2	Self-Adhesive Address Labels for Typewriters b...	14.620	2	
5	Eldon Expressions Wood and Plastic Desk Access...	48.860	7	
6	Newell 322	7.280	4	
7	Mitel 5320 IP Phone VoIP phone	907.152	6	
8	DXL Angle-View Binders with Locking Rings by S...	18.504	3	
...	...	...	...	
9986	Memorex Mini Travel Drive 64 GB USB 2.0 Flash ...	36.240	1	
9990	Tenex B1-RE Series Chair Mats for Low Pile Car...	91.960	2	
9991	Aastra 57i VoIP phone	258.576	2	
9992	It's Hot Message Books with Stickers, 2 3/4" x 5"	29.600	4	
9993	Acco 7-Outlet Masterpiece Power Center, Wihtou...	243.160	2	

	Discount	Profit
2	0.0	6.8714
5	0.0	14.1694
6	0.0	1.9656
7	0.2	90.7152
8	0.2	5.7825

```
#12
n = df[['Customer Name','Category','Sales','Profit']]
print(n.head(10))

#13
print(df.iloc[[0,5,10,15,20]])
```

```

-----
KeyError                                Traceback (most recent call last)
/tmp/ipykernel_903/224339109.py in <cell line: 0>()
    1 #12
----> 2 n = df[['Customer Name', 'Category', 'Sales', 'Profit']]
    3 print(n.head(10))
    4
    5 #13

```

```

#14
print(df.iloc[20:30])
#15
df.rename(columns={
    "Sales" : "Sales_Amount",
    "Profit" : "Profit_Amount",
    "Quantity" : "Quantity_Sold",
    "Category" : "Product_Category"
}, inplace=True)

print(df)

```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
20	21	CA-2014-143336	2014-08-27	2014-09-01	Second Class	ZD-21925	
21	22	CA-2016-137330	2016-12-09	2016-12-13	Standard Class	KB-16585	
22	23	CA-2016-137330	2016-12-09	2016-12-13	Standard Class	KB-16585	
23	24	US-2017-156909	2017-07-16	2017-07-18	Second Class	SF-20065	
24	25	CA-2015-106320	2015-09-25	2015-09-30	Standard Class	EB-13870	
25	26	CA-2016-121755	2016-01-16	2016-01-20	Second Class	EH-13945	
26	27	CA-2016-121755	2016-01-16	2016-01-20	Second Class	EH-13945	
27	28	US-2015-150630	2015-09-17	2015-09-21	Standard Class	TB-21520	
28	29	US-2015-150630	2015-09-17	2015-09-21	Standard Class	TB-21520	
29	30	US-2015-150630	2015-09-17	2015-09-21	Standard Class	TB-21520	

	Customer Name	Segment	Country	City	...	\
20	Zuschuss Donatelli	Consumer	United States	San Francisco	...	
21	Ken Black	Corporate	United States	Fremont	...	
22	Ken Black	Corporate	United States	Fremont	...	
23	Sandra Flanagan	Consumer	United States	Philadelphia	...	
24	Emily Burns	Consumer	United States	Orem	...	
25	Eric Hoffmann	Consumer	United States	Los Angeles	...	
26	Eric Hoffmann	Consumer	United States	Los Angeles	...	
27	Tracy Blumstein	Consumer	United States	Philadelphia	...	
28	Tracy Blumstein	Consumer	United States	Philadelphia	...	
29	Tracy Blumstein	Consumer	United States	Philadelphia	...	

	Postal Code	Region	Product ID	Product_Category	Sub-Category	\
20	94109	West	OFF-BI-10002215	Office Supplies	Binders	
21	68025	Central	OFF-AR-10000246	Office Supplies	Art	
22	68025	Central	OFF-AP-10001492	Office Supplies	Appliances	
23	19140	East	FUR-CH-10002774	Furniture	Chairs	
24	84057	West	FUR-TA-10000577	Furniture	Tables	
25	90049	West	OFF-BI-10001634	Office Supplies	Binders	
26	90049	West	TEC-AC-10003027	Technology	Accessories	
27	19140	East	FUR-BO-10004834	Furniture	Bookcases	
28	19140	East	OFF-BI-10000474	Office Supplies	Binders	
29	19140	East	FUR-FU-10004848	Furniture	Furnishings	

	Product Name	Sales_Amount	Quantity	\
20	Wilson Jones Hanging View Binder, White, 1"	22.720	4	
21	Newell 318	19.460	7	
22	Acco Six-Outlet Power Strip, 4' Cord Length	60.340	7	
23	Global Deluxe Stacking Chair, Gray	71.372	2	
24	Bretford CR4500 Series Slim Rectangular Table	1044.630	3	
25	Wilson Jones Active Use Binders	11.648	2	
26	Imation 8GB Mini TravelDrive USB 2.0 Flash Drive	90.570	3	
27	Riverside Palais Royal Lawyers Bookcase, Royal...	3083.430	7	
28	Avery Recycled Flexi-View Covers for Binding S...	9.618	2	
29	Howard Miller 13-3/4" Diameter Brushed Chrome ...	124.200	3	

	Discount	Profit_Amount
20	0.2	7.3840
21	0.0	5.0596
22	0.0	15.6884
23	0.3	-1.0196
24	0.0	240.2649
25	0.2	4.2224
26	0.0	11.7741
27	0.5	-1665.0522
28	0.7	-7.0532

```

#fixing data type - price should be numeric
df["price"] = df["price"].astype(float)
print(df)

```

Start coding or [generate](#) with AI.

